

Lecture Notes: Algorithmic Modeling and 2D Grid Manipulation

Computer Science Foundations

April 4, 2026

Abstract

This document explores the intersection of mathematical theory and practical algorithmic implementation for 2D grid manipulations. Taking the “Character Picture Grid” problem as a foundational case study, we examine how abstract mathematical matrices translate to data structures in memory, the nuances of row-major versus column-major traversal, and the asymptotic complexity of nested loop operations. We further extend these concepts to advanced matrix transformations, matrix multiplication, and graph theory models.

1 Introduction to Modeling

As defined in algorithm design, **modeling** is the art of abstracting a messy real-world application into a clean problem suitable for algorithmic attack. Proper modeling relates a specific application to known mathematical structures, such as **graphs**, **trees**, or **matrices**.

In the context of rendering a 2D image, our real-world object is a visual picture, and our mathematical model is a matrix. By mapping pixels to discrete matrix coordinates, we can apply rigorous algorithmic techniques to traverse, transform, and analyze the image.

2 Mathematical Foundation: The Matrix

A **matrix** A is a rectangular array of data where $a_{i,j}$ denotes the element appearing in the i -th row and j -th column. In the context of computer imagery, we model an image as a grid of **pixels** (picture elements).

For a matrix $A \in \mathbb{R}^{m \times n}$:

$$A = \begin{pmatrix} a_{0,0} & a_{0,1} & \cdots & a_{0,n-1} \\ a_{1,0} & a_{1,1} & \cdots & a_{1,n-1} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m-1,0} & a_{m-1,1} & \cdots & a_{m-1,n-1} \end{pmatrix}$$

2.1 Coordinate Systems: Cartesian vs. Screen

Computer graphics systems and terminal outputs model the origin $(0,0)$ in the **upper-left corner**, where:

- x -coordinates (columns) increase going **right**.
- y -coordinates (rows) increase going **down**.

3 Problem Analysis: Character Picture Grid

We are given a grid represented as a **list of lists** in Python. This data structure serves as a model for 2D space.

3.1 Modeling the Input Instance

The provided grid consists of 9 inner lists, each containing 6 strings. Mathematically:

- Number of columns (x -axis): $m = 9$
- Number of rows (y -axis): $n = 6$

In this specific model, `grid[x][y]` identifies the character at horizontal position x and vertical position y .

3.2 Challenge: Storage Layout vs. Display Layout

The data is stored in **column-major order**. This means that each inner list represents a vertical column of the image.

However, terminal windows output text in **row-major order** (line by line). To print the image correctly, the algorithm must iterate through y values in the outer loop and x values in the inner loop.

4 Implementation

```
1 grid = [['.', '.', '.', '.', '.', '.'],
2         ['.', '0', '0', '.', '.', '.'],
3         ['0', '0', '0', '0', '.', '.'],
4         ['0', '0', '0', '0', '0', '.'],
5         ['.', '0', '0', '0', '0', '0'],
6         ['0', '0', '0', '0', '0', '.'],
7         ['0', '0', '0', '0', '.', '.'],
8         ['.', '0', '0', '.', '.', '.'],
9         ['.', '.', '.', '.', '.', '.']]
10
11 for y in range(6):
12     for x in range(9):
13         print(grid[x][y], end='')
14     print()
```

Listing 1: Standard Heart Pattern Implementation

5 Additional Grid Instances (Practice Problems)

To master grid manipulation, we must be able to recognize patterns and calculate their dimensions regardless of the stored shape.

5.1 Example 1: The Chevron (Arrowhead)

Consider a grid stored as 5 columns, each 5 characters deep.

```
1 chevron = [['.', '.', 'X', '.', '.'],
2            ['.', 'X', 'X', '.', '.'],
3            ['X', 'X', 'X', '.', '.'],
4            ['.', 'X', 'X', '.', '.'],
5            ['.', '.', 'X', '.', '.']]
```

Listing 2: Chevron Pattern Input

Task: Render this column-major grid to the screen.

Analysis: Here $m = 5$ (columns) and $n = 5$ (rows).

```

1 for y in range(5):
2     for x in range(5):
3         print(chevron[x][y], end='')
4     print()

```

5.2 Example 2: The Digital '7'

Consider a grid representing a digital digit. This is stored as 4 columns of height 7.

```

1 seven = [['#', '.', '.', '.', '.', '.', '.', '.'],
2          ['#', '.', '.', '.', '.', '.', '.', '.'],
3          ['#', '.', '.', '.', '.', '.', '.', '.'],
4          ['#', '#', '#', '#', '#', '#', '#', '#']]

```

Listing 3: Digital Digit Seven

Task: Render this to the screen. Note the asymmetrical dimensions.

Analysis: $m = 4$ (columns), $n = 7$ (rows).

```

1 for y in range(7):
2     for x in range(4):
3         print(seven[x][y], end='')
4     print()

```

6 Advanced Algorithmic Transformations

6.1 Matrix Transposition

The transpose A^T is defined by $(A^T)_{i,j} = A_{j,i}$.

```

1 transposed_grid = [[grid[x][y] for x in range(9)] for y in range(6)]

```

Listing 4: Matrix Transposition

6.2 Matrix Multiplication

The product $C = AB$ where $C_{i,j} = \sum_{k=0}^{p-1} A_{i,k} \cdot B_{k,j}$.

```

1 def multiply_matrices(A, B):
2     m, p, n = len(A), len(A[0]), len(B[0])
3     C = [[0 for _ in range(n)] for _ in range(m)]
4     for i in range(m):
5         for j in range(n):
6             for k in range(p):
7                 C[i][j] += A[i][k] * B[k][j]
8     return C

```

Listing 5: Matrix Multiplication

7 Complexity Analysis

- **Traversal/Rotation:** Time complexity is $\Theta(V)$, where $V = n \times m$.
- **Matrix Multiplication:** For $n \times n$ matrices, time complexity is $\Theta(n^3)$.
- **Space Complexity:** $O(1)$ for in-place printing, $O(n \times m)$ for creating new transformed grids.

8 References

- Skiena, S.S. (2008) *The Algorithm Design Manual*, 2nd Edition.
- Cormen, T.H., et al. (2022) *Introduction to Algorithms*, 4th Edition.
- Sweigart, A. (2015) *Automate the Boring Stuff with Python*.
- Epp, S. S. (2010) *Discrete Mathematics with Applications*.